

Leva.Framework.Sample.LiveDashboard

```
1  using Leva.Framework.Core;
2  using Leva.Framework.Notifications;
3  using Leva.Framework.Notifications.Memory;
4  using Leva.Framework.Notifications.SignalR;
5  using Leva.Framework.Sample.LiveDashboard;
6  using Microsoft.AspNetCore.SignalR;
7  using Microsoft.Extensions.Hosting;
8
9  var builder = WebApplication.CreateBuilder(args);
10
11 // Sample host behavior: stop quickly even with an open SignalR connection.
12 builder.Services.Configure<HostOptions>(options =>
13 {
14     options.ShutdownTimeout = TimeSpan.FromSeconds(2);
15 });
16
17 builder.Services.AddSingleton<IClock, SystemClock>();
18 builder.Services.AddSingleton<IUserIdProvider, QueryStringUserIdProvider>();
19
20 // SignalR is the live delivery provider for framework notifications.
21 builder.Services.AddSignalRNotifications();
22
23 // The visible page history reads this concrete memory store.
24 builder.Services.AddSingleton<MemoryNotificationStore>();
25 builder.Services.AddSingleton<INotificationStore>(provider =>
26 provider.GetRequiredService<MemoryNotificationStore>());
27 builder.Services.AddSingleton<DashboardNotificationHistory>();
28
29 // NotificationService sends through SignalR and returns entries for the visible history.
30 builder.Services.AddSingleton<NotificationService>();
31
32 var app = builder.Build();
33
34 app.UseDefaultFiles();
35 app.UseStaticFiles(
36     new StaticFileOptions
37     {
38         OnPrepareResponse = context =>
39         {
40             // Samples change often while developing, so disable browser caching for local static
41             // files.
42             context.Context.Response.Headers.CacheControl = "no-store, no-cache, must-
43             revalidate";
44             context.Context.Response.Headers.Pragma = "no-cache";
45             context.Context.Response.Headers.Expires = "0";
46         },
47     },
48 );
49 app.MapGet("/favicon.ico", () => Results.NoContent());
50
51 // Expose the framework notification hub used by the browser page.
52 app.MapSignalRNotifications();
53
54 app.MapPost(
55     "/dashboard/notify",
56     async (NotificationService notifications, DashboardNotificationHistory history, CancellationToken
57     token) =>
58     {
59         var result = await SendAndRecordDemoNotificationAsync(notifications, history, "Live update",
60         token);
61         return result.IsSuccess
62             ? Results.Ok(DashboardNotificationEntry.From(result.Value!))
63             : Results.BadRequest(result.Error);
64     }
65 );
```

```

61
62 app.MapPost(
63     "/dashboard/self-check",
64     async (NotificationService notifications, DashboardNotificationHistory history, CancellationToken
65     token) =>
66     {
67         var before = history.Entries.Count;
68         var result = await SendAndRecordDemoNotificationAsync(notifications, history, "Self-check",
69         token);
70
71         var after = history.Entries.Count;
72         var passed = result.IsSuccess && after == before + 1;
73
74         return Results.Ok(
75             new DashboardSelfCheck(
76                 passed,
77                 passed ? "notification was sent through SignalR and stored"
78                     : result.IsFailure ? result.ErrorMessage
79                     : "notification was not stored as expected",
80                 before,
81                 after
82             )
83         );
84     }
85 );
86
87 app.MapGet(
88     "/dashboard/history",
89     (DashboardNotificationHistory history) =>
90     {
91         // The browser receives a DTO instead of raw framework NotificationEntry objects.
92         history.Snapshot()
93     }
94 );
95
96 app.Lifetime.ApplicationStarted.Register(() =>
97 {
98     Console.WriteLine("LiveDashboard sample");
99     Console.WriteLine(
100         "Open the printed local URL, then use the page buttons to send notifications and run the
101         self-check."
102     );
103     Console.WriteLine("Ctrl+C waits at most two seconds for active SignalR connections in this sample
104     host.");
105 });
106
107 app.Run();
108
109 static async Task<Result<NotificationEntry>> SendAndRecordDemoNotificationAsync(
110     NotificationService notifications,
111     DashboardNotificationHistory history,
112     string subject,
113     CancellationToken token
114 )
115 {
116     // Use the reusable notification service, then record the returned entry for the page.
117     var result = await notifications.SendAsync(
118         new NotificationRecipient("demo", DisplayName: "Demo user"),
119         SignalRNotificationOptions.DefaultChannel,
120         subject,
121         $"Server time is {DateTimeOffset.UtcNow:HH:mm:ss} UTC.",
122         token
123     );
124
125     if (result.IsSuccess)
126         await history.RecordAsync(result.Value!, token);
127
128     return result;
129 }
130

```